

Plano de Execução — Projeto A3: Conformidade de Contrato da API REST

Sep 26, 2026 · @Matheus Henrique Dias Siqueira

O projeto aplica um processo completo de garantia da qualidade sobre a Order Management API (Java 25 + Spring Boot 4.1 + PostgreSQL) para provar, com evidências reproduzíveis, que a API cumpre o contrato OpenAPI: Bean Validation, formato padronizado de erro (400/404/409/500) e consistência dos DTOs. Entrega final até Nov 30, 2026 ; são 9 semanas de trabalho a partir de 28/09.

1. Decisões iniciais e alertas

A equipe testa a API existente; o objetivo não é desenvolver a aplicação, e sim produzir evidências de qualidade sobre ela (seção 2 do enunciado).

Decisões propostas

Tema	Decisão	Justificativa
Sistema sob teste (SUT)	Order Management API (repositório JkbSousa/order-management-api), fork na conta da equipe	Já possui DTOs, Bean Validation, handler global de erros e Swagger — exatamente o tema de conformidade de contrato
Linguagem dos testes	Java 25, a mesma do SUT	Caixa-branca e cobertura exigem instrumentar o código Java; testes em Python não medem cobertura do código Spring
Complemento de contrato	Schemathesis (CLI, sem programar em Python)	Gera centenas de requisições a partir do OpenAPI e verifica se as respostas respeitam o contrato
Banco de testes	PostgreSQL real em container (Testcontainers)	Testa constraints reais (UNIQUE de CPF, FK) e atende também à disciplina de Banco de Dados
Gestão de defeitos	GitHub Issues + GitHub Projects	Rastreabilidade Requisito → Caso de teste → Resultado → Defeito com links clicáveis

Tema	Decisão	Justificativa
Pipeline	GitHub Actions	Executa build, análise estática, testes e cobertura a cada push, como pede a seção 14

Alertas que precisam de resposta antes de 04/10

- ☐ **Tamanho do grupo:** o enunciado limita a 3 alunos por grupo; a equipe tem 4. Confirmar com o professor Christian Azonyetin por escrito.
- ☐ **Requisitos de Banco de Dados:** o PDF recebido cobre apenas Garantia da Qualidade. A trilha de BD (seção 9 deste plano) é uma proposta; validar com o professor de BD o que ele exige.
- ☐ **Stack diferente da recomendada:** o professor recomenda Python + pytest, mas não obriga linguagem (seção 21). Informar que o grupo usará Java + JUnit 5 + JaCoCo por coerência com o SUT.

2. Diagnóstico do sistema sob teste

A API tem 5 recursos, 28 endpoints e apenas 1 teste (`contextLoads`), ou seja, cobertura praticamente zero: o ponto de partida ideal para a comparação antes × depois.

Recurso	Rotas	Regras de negócio relevantes	Risco
Orders	<code>GET/POST /orders</code> , <code>GET/DELETE /orders/{id}</code> , <code>PUT /orders/{id}/status</code> , <code>PUT /orders/{id}/address</code>	Cria cliente, itens, pagamento e endereço numa transação; endereço bloqueado após SHIPPED/DELIVERED	Crítico
Payment	<code>GET/POST /payment</code> , <code>GET/PUT/DELETE /payment/{id}</code> , <code>POST /payment/{id}/process</code>	Edição só em PENDING; troca de tipo (CARD/PIX/BOLETO) recria o registro; 1 pagamento por pedido	Crítico
Clients	<code>CRUD /clients</code> , <code>POST /clients/{id}/address</code>	INDIVIDUAL exige CPF, CORPORATE exige CNPJ; CPF/CNPJ únicos	Alto
Products	<code>CRUD /products</code>	PHYSICAL exige peso, DIGITAL exige link; preço positivo	Médio
Address	<code>CRUD /address</code>	CEP com 8 dígitos, UF com 2 caracteres	Baixo

Arquitetura: controllers → services → repositories (Spring Data JPA) → PostgreSQL, com herança `SINGLE_TABLE` em Client, Product e Payment e `@RestControllerAdvice` para erros.

Defeitos candidatos encontrados na leitura do código. São hipóteses: só viram defeito registrado depois que um teste executado as confirmar (regra 25 do enunciado).

ID	Onde	Problema	Violação de contrato esperada
DC-01	<code>pom.xml</code>	springdoc 2.8.5 com Spring Boot 4.1; a linha compatível com Boot 4 é a 3.x	Swagger/ <code>/v3/api-docs</code> pode não subir — verificar na Fase 0
DC-02	<code>GlobalExceptionHandler.handleGenericException</code>	Corpo diz <code>status: 400</code> , HTTP devolve 500	Corpo e status HTTP divergem
DC-03	<code>handleDataIntegrityViolation</code>	Toda violação de integridade responde "This order already has a payment."	CPF duplicado ou exclusão com FK retornam mensagem falsa
DC-04	Handler global	JSON malformado, <code>id</code> não numérico e método não suportado caem no handler genérico	Retorna 500 onde o correto é 400/405
DC-05	<code>ClientService</code>	Erros de cliente lançam <code>ProductException</code>	Campo <code>error</code> = "Product Error" em rota de cliente
DC-06	Todos os <code>POST</code>	Criação retorna 200	Padrão REST é 201 + <code>Location</code>
DC-07	<code>DELETE</code> de <code>clients/products/orders</code>	Retorna 200 vazio; <code>address/payment</code> retornam 204; <code>id</code> inexistente não gera 404	Inconsistência entre recursos
DC-08	<code>POST /clients/{id}/address</code>	Recebe a entidade <code>Address</code> sem <code>@Valid</code>	Bean Validation contornada
DC-09	DTOs	<code>type</code> é String livre; CPF/CNPJ sem validação de formato	Aceita CPF "abc"

ID	Onde	Problema	Violação de contrato esperada
DC-10	<code>OrderService.updateStatus</code>	Sem máquina de estados	DELIVERED pode voltar a PENDING_PAYMENT
DC-11	<code>PaymentService.processAndSave</code>	Marca o pedido como PAID sem checar resultado nem status atual	Pagamento reprocessado ou recusado deixa pedido pago
DC-12	<code>OrderService.insert</code>	<code>payment.amount</code> não é comparado ao total do pedido	Pedido de R\$ 450 pago com R\$ 1
DC-13	<code>ErrorResponse.details</code>	Lista só mensagens, sem o nome do campo	Cliente da API não sabe qual campo corrigir
DC-14	Controllers	Sem <code>@ApiResponse</code> ; o Swagger não documenta 400/404/409	O contrato publicado está incompleto
DC-15	<code>application.properties</code>	Senha em texto puro e <code>ddl-auto=update</code>	Segurança e controle de esquema (BD)
DC-16	Entidades	Valores monetários em <code>Double</code>	Erro de arredondamento em totais

3. Equipe e papéis

Gabriel lidera a parte técnica e revisa todo código antes do merge; cada frente tem um responsável único (R) e um apoio (A). A distribuição de Allan e Francisco é uma proposta — ajustar na reunião de kickoff conforme a afinidade de cada um.

Integrante	RA	Papel	Responsabilidades principais (R)	Apoio (A)
Gabriel Sousa Vasconcelos	13526110176	Líder técnico / arquiteto de testes	Ambiente, pipeline CI, base de testes (Testcontainers, REST Assured), testes de integração e caixa-branca, correções no código, revisão de PRs	Todas as frentes de código

Integrante	RA	Papel	Responsabilidades principais (R)	Apoio (A)
Matheus Henrique Dias Siqueira	1352623492	Dados, métricas e Banco de Dados	Trilha de BD (Flyway, constraints, consultas), testes de repositório, coleta de métricas, dashboard e relatório de métricas, comparação antes × depois	Automação de relatórios no CI
Allan Gabriel Almeida Barros	1352524052	Analista de requisitos e testes caixa-preta	Requisitos (RF/RNF), critérios de aceitação, validação de requisitos, catálogo de casos de teste, testes de sistema e aceitação	Registro de defeitos
Francisco Miguel de Lemos Sena Ferreira	1352523939	Analista de qualidade e documentação	Plano de Qualidade, revisão técnica formal, análise estática (antes/depois), gestão de defeitos, Schemathesis, ética e segurança, consolidação do relatório final	Plano de testes

Regras de trabalho

- Todo integrante escreve pelo menos 10 testes automatizados: na apresentação o professor pode pedir a qualquer um que crie ou execute um teste (regra 25).
- Nenhum merge em `main` sem PR aprovado por outra pessoa e pipeline verde.
- Reunião semanal de 30 min (domingo à noite) para atualizar o quadro e as métricas.

4. Stack e ferramentas

Toda a suíte roda com um único comando, `./mvnw verify`, localmente e no GitHub Actions.

Categoria	Ferramenta	Linguagem	Uso no projeto
Build	Maven (wrapper <code>mvnw</code>)	—	Compila, executa testes e gera relatórios
Testes unitários	JUnit 5 + Mockito + AssertJ	Java	Services isolados, com repositórios mockados
Testes de API (integração/sistema)	Spring Boot Test + REST Assured	Java	Requisições HTTP reais contra a API em porta aleatória
Banco nos testes	Testcontainers (PostgreSQL)	Java	Banco real e descartável por execução; exige Docker
Validação de contrato	swagger-request-validator (Atlassian) + REST Assured	Java	Falha o teste se requisição ou resposta divergir do <code>openapi.json</code>
Fuzzing de contrato	Schemathesis	CLI (via Docker)	Gera casos a partir do OpenAPI; detecta 500 e respostas fora do esquema
Cobertura (caixa-branca)	JaCoCo 0.8.14+	—	Cobertura de instruções e branches; versão com suporte oficial a Java 25
Análise estática	PMD + Checkstyle + SpotBugs (plugins Maven)	—	Bugs, estilo, código morto, complexidade
Qualidade consolidada	SonarCloud (gratuito para repositório público)	—	Duplicação, complexidade, vulnerabilidades, histórico antes × depois
Testes manuais/exploratórios	Swagger UI + Postman ou Insomnia	—	Exploração inicial e evidências de reprodução de defeitos

Categoria	Ferramenta	Linguagem	Uso no projeto
Banco de Dados	PostgreSQL 18 + Flyway + DBeaver	SQL	Migrações versionadas, constraints, consultas e modelagem
CI	GitHub Actions	YAML	Pipeline: build → estática → testes → cobertura → artefatos
Relatório de testes	Surefire/Failsafe + Allure (opcional)	—	PASS/FAIL por caso, com histórico
Métricas	Planilha ou Python/pandas lendo os XML do JaCoCo e Surefire	Python (opcional)	Consolidação e gráficos do relatório de métricas
Gestão	GitHub Issues + Projects	—	Defeitos, tarefas e rastreabilidade

Pré-requisitos na máquina de cada integrante: JDK 25 (Temurin), Docker Desktop, Git, IntelliJ IDEA Community ou VS Code com Extension Pack for Java, Postman/Insomnia e DBeaver.

5. Estrutura do repositório e fluxo de trabalho

O enunciado pede `/src`, `/tests`, `/docs`, `/reports`, `/config` e `README.md`; em Maven os testes ficam em `src/test`, o que o próprio enunciado aceita ("a estrutura poderá variar conforme a tecnologia").

```

order-management-api/
├─ src/main/java/...          # SUT (código da API)
├─ src/main/resources/db/migration/ # scripts Flyway (V1__schema.sql,
V2__seed.sql)
├─ src/test/java/.../unit/      # testes unitários (services)
├─ src/test/java/.../integration/ # repositórios + Testcontainers
├─ src/test/java/.../api/       # sistema, contrato, aceitação,
regressão (REST Assured)
├─ docs/                        # plano de qualidade, plano de testes,
requisitos, catálogo, relatórios
├─ reports/                     # snapshots: jacoco, pmd, spotbugs,
schemathesis, métricas (antes/depois)
├─ config/                      # checkstyle.xml, pmd-ruleset.xml,
openapi.json congelado
├─ .github/workflows/ci.yml
└─ README.md                    # como rodar tudo com um comando

```

Fluxo Git

1. Branch `main` protegida; trabalho em branches `feature/<fase>-<tema>` ou `fix/D-xxx`.
2. Commits em Conventional Commits (`test: add boundary tests for zip code`).
3. Cada defeito vira uma Issue `D-xxx` ; o PR de correção referencia a Issue (`fixes #12`) e inclui o teste de regressão.
4. Criar a tag `v0-baseline` antes de qualquer correção: é a versão "antes" da comparação de melhoria contínua.

6. Fases 0 a 4 — preparação, requisitos, planejamento e análise

Essas fases produzem os documentos que valem 31 dos 100 pontos e definem tudo o que será testado depois; nenhum teste deve ser escrito antes de a Fase 1 estar aprovada.

Fase 0 — Preparação do ambiente

Responsável: Gabriel · **Dificuldade:** Média · **Esforço:** 10 h

1. Fazer fork do repositório, proteger `main` , adicionar os 4 integrantes.
2. Todos instalam os pré-requisitos e sobem a API: `docker run ... postgres` + `./mvnw spring-boot:run` .
3. Abrir `http://localhost:8080/swagger-ui/index.html` e `/v3/api-docs` . Se não subir, registrar DC-01 como o primeiro defeito real e corrigir atualizando springdoc para a linha 3.x.
4. Mover senha do banco para variáveis de ambiente (`${DB_PASSWORD}`).

5. Salvar o `openapi.json` gerado em `config/openapi-v0.json` e criar a tag `v0-baseline`.
6. Criar o GitHub Project com colunas Backlog · Em andamento · Em revisão · Concluído, e os labels `defeito`, `severidade:alta/média/baixa`, `prioridade:*`.

Entregável: API rodando nas 4 máquinas, contrato congelado, quadro configurado.

Fase 1 — Definição do sistema, requisitos e critérios de aceitação

Responsável: Allan · **Apoio:** Gabriel · **Dificuldade:** Média · **Esforço:** 14 h

O mínimo exigido é 10 RF e 5 RNF; a meta é 12 RF e 6 RNF, todos derivados do código e do Swagger. Rascunho inicial:

ID	Requisito	Exemplos de critério de aceitação
RF-01	Cadastrar cliente PF ou PJ	CA: rejeitar nome vazio (400); exigir CPF para INDIVIDUAL; rejeitar CPF duplicado (409)
RF-02	Consultar, atualizar e remover cliente	CA: id inexistente → 404 no formato padrão
RF-03	Cadastrar produto físico ou digital	CA: preço $\leq 0 \rightarrow 400$; PHYSICAL sem peso $\rightarrow 400$
RF-04	Consultar, atualizar e remover produto	CA: 404 padronizado
RF-05	Cadastrar endereço	CA: CEP com 7 ou 9 dígitos $\rightarrow 400$; UF com 1 ou 3 letras $\rightarrow 400$
RF-06	Criar pedido completo	CA: sem itens $\rightarrow 400$; produto inexistente $\rightarrow 404$; sem clientId e sem client $\rightarrow 400$
RF-07	Calcular total do pedido	CA: total = Σ (preço $\times$ quantidade) com 2 casas decimais
RF-08	Atualizar status do pedido	CA: status inválido $\rightarrow 400$; transições só para frente
RF-09	Alterar endereço de entrega	CA: bloqueado em SHIPPED/DELIVERED $\rightarrow 400$
RF-10	Registrar pagamento CARD/PIX/BOLETO	CA: campo obrigatório do tipo ausente $\rightarrow 400$; segundo pagamento no pedido $\rightarrow 409$
RF-11	Editar pagamento	CA: só em PENDING; troca de tipo preserva o vínculo com o pedido

ID	Requisito	Exemplos de critério de aceitação
RF-12	Processar pagamento	CA: pedido só vira PAID se o pagamento for APPROVED
RNF-01	Contrato de erro padronizado	Todo 4xx/5xx retorna <code>timestamp</code> , <code>status</code> , <code>error</code> , <code>message</code> , <code>details</code> , e <code>status</code> do corpo = status HTTP
RNF-02	Conformidade OpenAPI	100% das respostas validadas contra o <code>openapi.json</code>
RNF-03	Desempenho	95% das consultas <code>GET</code> respondem em até 500 ms com 50 registros na base
RNF-04	Segurança	Credenciais fora do código; número de cartão sempre mascarado na resposta
RNF-05	Manutenibilidade	Complexidade ciclomática ≤ 10 por método; 0 bugs de severidade alta no SpotBugs
RNF-06	Portabilidade	Execução idêntica em Windows, Linux e CI via Docker

Entregável: `docs/requisitos.md` com descrição do sistema, problema, público-alvo, funcionalidades, RF, RNF, restrições, riscos e critérios CA-xx por requisito.

Fase 2 — Validação dos requisitos, Plano de Qualidade e Plano de Testes

Responsável: Francisco · **Apoio:** Allan · **Dificuldade:** Média · **Esforço:** 16 h

- Validar requisitos:** revisar cada RF/RNF com um checklist (ambíguo, incompleto, contraditório, não verificável, sem CA, termo subjetivo). Registrar os problemas e a reescrita. Exemplo: "a API deve responder rápido" → RNF-03.
- Plano de Qualidade** (`docs/plano-qualidade.md`) com os 11 itens da seção 4.2: objetivos, atributos (confiabilidade, eficiência, manutenibilidade, portabilidade, segurança; usabilidade adaptada como "clareza das mensagens de erro"), critérios de aceitação, estratégia, níveis, ferramentas, responsabilidades, riscos, critérios de entrada, saída e aprovação.
- Critérios de saída sugeridos:** $\geq 95\%$ dos casos aprovados, 0 defeitos críticos abertos, cobertura de branches $\geq 80\%$ nos services, 0 violações de contrato no Schemathesis.
- Matriz de risco** (probabilidade $\times$ impacto) por funcionalidade, justificando quantos casos cada uma recebe: Pedido e Pagamento críticos, Cliente alto, Produto médio, Endereço baixo.

5. **Plano de Testes** (docs/plano-testes.md): escopo, itens fora de escopo (autenticação, carga pesada), abordagem por nível, ambiente, dados de teste, cronograma.

Entregável: plano de qualidade, plano de testes e relatório de validação de requisitos.

Fase 3 — Revisão técnica formal (inspeção)

Responsável: Francisco (moderador) · **Participantes:** todos · **Dificuldade:** Baixa · **Esforço:** 8 h

1. Definir papéis: moderador (Francisco), autor (Gabriel, que conhece o código), revisores (Allan e Matheus), escriba.
2. Cada revisor analisa previamente requisitos, arquitetura, código (controllers, services, handler), casos de teste e README com checklist.
3. Reunião de 1 h gravada ou com ata; cada achado vira registro D-xxx com artefato, descrição, severidade, prioridade e status.
4. A tabela de defeitos candidatos (seção 2) é o insumo inicial.

Entregável: ata da inspeção + defeitos registrados nas Issues.

Fase 4 — Análise estática (linha de base)

Responsável: Francisco · **Apoio:** Gabriel · **Dificuldade:** Média · **Esforço:** 10 h

1. Adicionar ao pom.xml os plugins maven-pmd-plugin , maven-checkstyle-plugin e spotbugs-maven-plugin , com regras em config/ .
2. Conectar o repositório ao SonarCloud e rodar a análise pelo GitHub Actions.
3. Executar na tag v0-baseline e salvar em reports/static-before/ : total de issues por categoria (bugs, code smells, vulnerabilidades, duplicação %, complexidade).
4. Classificar os achados; os relevantes viram defeitos. A segunda execução ocorre na Fase 10.

Entregável: relatório de análise estática “antes”.

7. Fases 5 a 8 — casos de teste e automação

Este é o núcleo técnico: 42 dos 100 pontos (caixa-preta 10, caixa-branca 10, níveis 10, automação 12). A meta é cerca de 80 testes automatizados, bem acima do mínimo de 30 casos.

Fase 5 — Projeto dos casos de teste (caixa-preta)

Responsável: Allan · **Apoio:** todos · **Dificuldade:** Média · **Esforço:** 16 h

Cada caso no catálogo (docs/catalogo-casos.md ou planilha) tem: ID, requisito, objetivo, pré-condições, dados de entrada, passos, resultado esperado, resultado obtido, status e evidência (link do relatório do CI). Usar 6 técnicas (o mínimo são 3):

Técnica	Aplicação na API	Casos-exemplo
Particionamento de equivalência	type de cliente, produto e pagamento	Válidas: INDIVIDUAL, CORPORATE; inválidas: "", "XYZ", null
Análise de valor-limite	CEP (8 dígitos), UF (2), preço (> 0), quantidade (> 0), peso (≥ 0), data de nascimento (passado)	CEP 7/8/9 dígitos; preço -0.01/0/0.01; quantidade 0/1; nascimento ontem/hoje/amanhã
Tabela de decisão	Pagamento: tipo × campo obrigatório presente; pedido: status × alteração de endereço	PIX sem pixKey → 400; SHIPPED + novo endereço → 400; PROCESSING + novo endereço → 200
Baseado em requisitos	Um ou mais casos por CA-xx	Rastreabilidade RF → CT na matriz
Baseado em risco	Pedido e Pagamento recebem mais casos	Justificativa ligada à matriz da Fase 2
Casos inválidos e de exceção	JSON malformatado, id não numérico, método errado, recurso inexistente, CPF duplicado	Esperado: 400/404/405/409 no formato padrão, nunca 500

Fase 6 — Infraestrutura de automação e pipeline

Responsável: Gabriel · **Dificuldade:** Alta · **Esforço:** 16 h

1. Adicionar dependências de teste: rest-assured, testcontainers (postgresql, junit-jupiter), swagger-request-validator-restassured, assertj.
2. Criar classe base ApiTestBase com @SpringBootTest(webEnvironment = RANDOM_PORT), container PostgreSQL compartilhado e filtro de validação OpenAPI aplicado a toda requisição.
3. Criar builders de dados (ClientFixture, OrderFixture) para evitar JSON repetido.
4. Configurar JaCoCo (prepare-agent + report) e Surefire/Failsafe; separar testes com @Tag("unit"), @Tag("integration"), @Tag("api"), @Tag("regression").

5. Criar `.github/workflows/ci.yml`: checkout → JDK 25 → `./mvnw verify` → PMD/Checkstyle/SpotBugs → SonarCloud → upload de `target/site/jacoco` e relatórios Surefire como artefatos.

Exemplo do padrão de teste de API que todos seguirão:

```
@Test
@Tag("api")
void ct017_deveRejeitarCepCom7Digitos() {
    given().filter(openApiValidation)           // valida contra o contrato
        .contentType(JSON).body(AddressFixture.comCep("50000000"))
        .when().post("/address")
        .then().statusCode(400)
            .body("status", equalTo(400))        // RNF-01: corpo = HTTP
            .body("error", equalTo("Validation Error"))
            .body("details", hasItem("ZIP code must contain 8 digits.));
}
```

Fase 7 — Testes por nível

Responsáveis: conforme tabela · **Dificuldade:** Média a Alta · **Esforço:** 40 h (somado)

Nível	O que testar	Como	Responsável	Meta
Unitário	Regras dos 5 services: <code>buildClient</code> , <code>buildPayment</code> , <code>updateStatus</code> , <code>updateAddress</code> , update de pagamento	JUnit 5 + Mockito, sem Spring e sem banco	Gabriel e Matheus	25 testes
Integração	Repositories + mapeamento JPA + constraints (UNIQUE de CPF, FK, cascade, orphanRemoval)	<code>@DataJpaTest</code> + Testcontainers	Matheus	10 testes
Sistema	Endpoints ponta a ponta via HTTP, incluindo todos os casos de erro	REST Assured + validador OpenAPI	Allan e Francisco	30 testes

Nível	O que testar	Como	Responsável	Meta
Contrato	Todas as respostas conformes ao <code>openapi.json</code> ; fuzzing	Filtro do validador em todos os testes de API + Schemathesis <code>run <url>/v3/api-docs --checks all</code>	Francisco	0 violações
Aceitação	Um cenário por RF no formato Dado/Quando/Então (ex.: fluxo pedido → pagamento → processamento → PAID)	REST Assured com <code>@DisplayName</code> em linguagem de negócio	Allan	12 testes
Regressão	Todo defeito corrigido ganha um teste que falhava antes e passa depois	<code>@Tag("regression");</code> suíte completa roda a cada push	Quem corrigiu	1 por defeito

Fase 8 — Testes caixa-branca e cobertura

Responsável: Gabriel · **Apoio:** Matheus · **Dificuldade:** Alta · **Esforço:** 14 h

1. Rodar JaCoCo após as Fases 6–7 e registrar cobertura de instruções e branches por classe ("cobertura antes").
2. Escolher 3 métodos de maior complexidade (`PaymentService.update` , `OrderService.insert` , `GlobalExceptionHandler`) e desenhar o grafo de fluxo de controle, calcular a complexidade ciclomática e listar os caminhos independentes.
3. Criar os casos que faltam para cobrir cada branch não exercitado (ex.: `typeChanged` verdadeiro para cada tipo; status diferente de PENDING).
4. Rodar novamente e registrar "cobertura depois", mostrando as linhas vermelhas → verdes no relatório HTML.
5. Interpretar: o que falta cobrir, por quê, e o risco residual (exigência da seção 16).

Metas: instruções $\geq 85\%$ e branches $\geq 80\%$ em `services` e `exceptions`; entidades e DTOs podem ficar abaixo, com justificativa.

8. Fases 9 a 12 — defeitos, melhoria contínua, métricas e fechamento

Essas fases transformam resultados de teste em decisão de engenharia e valem 20 pontos (defeitos 5, métricas 7, melhoria 4, ética 2, documentação 2), além de sustentar toda a apresentação.

Fase 9 — Gestão de defeitos

Responsável: Francisco · **Apoio:** Allan · **Dificuldade:** Baixa · **Esforço:** 8 h

1. Criar template de Issue com os 13 campos exigidos: ID, título, descrição, passos, resultado esperado, resultado obtido, severidade, prioridade, evidência, responsável, status, correção, reteste.
2. Todo teste que falha gera uma Issue `D-xxx` com link para o caso `CT-xxx` e para o log do CI.
3. Ciclo obrigatório, refletido nas colunas do quadro: Registrado → Em análise → Em correção → Em reteste → Regressão OK → Encerrado.
4. Meta realista: 12 a 18 defeitos reais registrados (a seção 2 já aponta 16 candidatos).

Fase 10 — Correções, reteste, regressão e nova análise estática

Responsável: Gabriel · **Apoio:** todos (cada um corrige ao menos 2 defeitos) · **Dificuldade:** Média · **Esforço:** 20 h

1. Priorizar por severidade × risco. Correções típicas: handler para `HttpMessageNotReadableException`, `MethodArgumentTypeMismatchException` e `HttpRequestMethodNotSupportedException`; status do corpo igual ao HTTP no 500; mensagem de 409 por causa real; `201 Created` nos POST; 404 no DELETE de id inexistente; `@ApiResponse` com o esquema `ErrorResponse`; validação de CPF/CNPJ; máquina de estados do pedido; `BigDecimal` para dinheiro.
2. Toda correção que muda o contrato atualiza o `openapi.json` e é registrada no relatório (contrato v0 → v1).
3. Reteste do caso que falhou + suíte de regressão completa no CI.
4. Nova rodada de PMD/Checkstyle/SpotBugs/SonarCloud → `reports/static-after/`.
5. Deixar 1 ou 2 defeitos de baixa severidade abertos e justificados: é realista e o enunciado mostra exemplo com status "Aberto".

Fase 11 — Métricas, atributos de qualidade e comparação antes × depois

Responsável: Matheus · **Dificuldade:** Média · **Esforço:** 14 h

1. Extrair automaticamente os números dos XML do Surefire (testes), JaCoCo (cobertura), PMD/SpotBugs e SonarCloud (complexidade, duplicação), e das Issues (defeitos). Um script simples ou planilha evita números digitados à mão.
2. Calcular: casos criados, executados, aprovados, reprovados, taxa de aprovação e de falha; defeitos encontrados, por severidade, corrigidos, abertos; densidade de defeitos = defeitos ÷ KLOC (o código principal tem cerca de 2,3 mil linhas não vazias); cobertura, complexidade, duplicação, problemas estáticos.
3. Mapear atributos a indicadores: confiabilidade → taxa de falhas e respostas 500 no Schemathesis; eficiência → tempo de resposta (RNF-03); manutenibilidade → complexidade e duplicação; segurança → vulnerabilidades e dados sensíveis expostos; portabilidade → execução em Windows, Linux e CI; usabilidade da API → % de erros com mensagem e campo identificáveis.
4. Montar a tabela antes × depois (tag `v0-baseline` × versão final) com cobertura, falhas, defeitos críticos, complexidade e problemas estáticos.
5. Interpretar cada número em 2–3 frases: o que significa, se é suficiente e qual risco resta.

Fase 12 — Ética, relatório final e apresentação

Responsável: Francisco (consolidação) · **Participantes:** todos · **Dificuldade:** Baixa · **Esforço:** 18 h

1. **Ética e segurança:** a API armazena CPF, CNPJ, e-mail, data de nascimento, chave PIX e número de cartão. Discutir LGPD (minimização, finalidade), o fato de o número completo do cartão ser persistido (PCI DSS proíbe armazenar dados sensíveis de autenticação e exige proteção do PAN), ausência de autenticação, senha no repositório e a pergunta obrigatória: consequências de uma falha grave (cobrança indevida, pedido pago sem pagamento, vazamento de dados).
2. **Relatório técnico final** (`docs/relatorio-final.md` ou PDF): resumo, SUT, processo aplicado, resultados por fase, métricas interpretadas, antes × depois, lições aprendidas, resposta à pergunta do enunciado.
3. **Apresentação** de 15–20 min com demonstração ao vivo (roteiro na seção 13).

9. Trilha de Banco de Dados

A trilha tira o esquema do controle implícito do Hibernate (`ddl-auto=update`) e o transforma em artefato versionado, validado e testado — o que serve à disciplina de BD e gera evidências para a de Qualidade. Confirmar o escopo com o professor de BD antes de 04/10.

Responsável: Matheus · **Apoio:** Gabriel · **Dificuldade:** Média · **Esforço:** 22 h

Etapa	O que fazer	Ferramenta	Entregável
B1. Engenharia reversa	Gerar o esquema atual a partir da API rodando e desenhar os modelos conceitual (DER) e lógico das 6 tabelas: <code>client</code> , <code>product</code> , <code>payment</code> , <code>orders</code> , <code>order_item</code> , <code>address</code>	DBeaver (ER Diagram) ou brModelo	<code>docs/bd/modelo-conceitual.png</code> , <code>modelo-logico.png</code>
B2. Análise do modelo	Avaliar normalização (1FN-3FN), o custo do <code>SINGLE_TABLE</code> (colunas nulas por subtipo) versus <code>JOINED</code> , e a desnormalização intencional de <code>order_item.price</code> (histórico de preço)	Texto técnico	<code>docs/bd/analise-modelo.md</code>
B3. Migrações	Adicionar Flyway; <code>V1__schema.sql</code> com PK, FK, NOT NULL, UNIQUE, <code>CHECK (price > 0, quantity > 0, status IN (...), CPF obrigatório quando dtype = 'INDIVIDUAL')</code> e índices nas FKs; trocar para <code>ddl-auto=validate</code>	Flyway + SQL PostgreSQL	Esquema versionado e reproduzível
B4. Dados de teste	<code>V2__seed.sql</code> com massa controlada (≥ 50 registros) para testes de sistema e para medir RNF-03	SQL	Base determinística

Etapa	O que fazer	Ferramenta	Entregável
B5. Consultas	6 a 8 consultas de negócio: faturamento por período, produtos mais vendidos, pedidos por status, clientes sem pedidos, pagamentos pendentes, ticket médio; 2 views; <code>EXPLAIN ANALYZE</code> antes e depois de um índice	SQL + DBeaver	<code>docs/bd/consultas.sql</code> + análise de plano
B6. Testes de BD	Provar que o banco rejeita dado inválido mesmo sem a API (inserção direta via <code>JdbcTemplate</code>), que a migração aplica em base vazia e que <code>cascade/orphanRemoval</code> funcionam	JUnit + Testcontainers	8 a 10 testes de integração
B7. Opcional (diferencial)	Esquema <code>qa</code> com tabelas <code>test_run</code> , <code>test_result</code> e <code>defect</code> , alimentadas pelo CI, e consultas SQL que geram as métricas da Fase 11	SQL + script	Ponte explícita entre as duas disciplinas

Uma constraint criada em B3 pode quebrar dados ou testes existentes; isso é esperado e deve ser registrado como defeito de modelo, não escondido.

10. Cronograma semanal

São 9 semanas com 3 marcos de controle; a última semana é reservada para relatório, ensaio e imprevistos.

Semana	Período	Fases	Entregas da semana	Marco
S1	28/09 – 04/10	F0, F1 (início)	API rodando nas 4 máquinas; tag <code>v0-baseline</code> ; alertas respondidos pelo professor	—

Semana	Período	Fases	Entregas da semana	Marco
S2	05/10 – 11/10	F1, F2, B1-B2	Requisitos, validação, Plano de Qualidade, Plano de Testes, matriz de risco, DER	M1: documentação de planejamento aprovada
S3	12/10 – 18/10	F3, F4, F5, B3	Ata da inspeção, análise estática “antes”, catálogo com ≥ 60 casos, migrações Flyway	—
S4	19/10 – 25/10	F6, F7 (unitário), B4	Pipeline CI verde, ApiTestBase , 25 testes unitários, massa de dados	—
S5	26/10 – 01/11	F7 (integração e sistema), B5-B6, F9	10 testes de integração, 30 de sistema, consultas SQL, defeitos registrados	—
S6	02/11 – 08/11	F7 (contrato e aceitação), F8 (antes)	Validador OpenAPI em toda a suíte, Schemathesis, 12 de aceitação, cobertura “antes”	M2: suíte completa; congelamento de novos casos
S7	09/11 – 15/11	F10, F8 (depois)	Correções, retestes, regressão, cobertura “depois”	—
S8	16/11 – 22/11	F4 (depois), F11, F12	Análise estática “depois”, relatório de métricas, antes × depois, seção de ética, rascunho do relatório final	M3: rascunho completo revisado
S9	23/11 – 30/11	F12	Relatório final, slides, 2 ensaios da demonstração, entrega até 30/11	Entrega

11. Esforço, dificuldade e peso na nota

O esforço total estimado é 226 h, cerca de 56 h por pessoa ou 6–7 h por semana; com 15% de margem, planejar 260 h. As fases de maior peso na nota são também as de maior dificuldade técnica, por isso concentram o apoio de Gabriel.

Fase	Dificuldade	Esforço (h)	Responsável	Critério da rubrica	Pontos
F7 Testes por nível	Média a Alta	40	Todos	Unitário, integração, sistema e aceitação	10
F6 Infra de automação e CI	Alta	16	Gabriel	Testes automatizados	12
F5 Casos caixa-preta	Média	16	Allan	Testes caixa-preta	10
F8 Caixa-branca	Alta	14	Gabriel	Testes caixa-branca	10
F1 Requisitos	Média	14	Allan	Requisitos e critérios de aceitação	8
F2 Planos e risco	Média	16	Francisco	Plano de qualidade + planeamento + risco	8 + 8 + 7
F3 + F4 Revisão e estática	Baixa / Média	18	Francisco	Análise estática e revisão técnica	7
F11 Métricas	Média	14	Matheus	Métricas e indicadores	7
F9 Defeitos	Baixa	8	Francisco	Gestão de defeitos	5
F10 Correções e regressão	Média	20	Gabriel + todos	Melhoria contínua	4
F12 Ética, relatório, apresentação	Baixa	18	Francisco + todos	Ética (2) + documentação (2)	4
F0 Ambiente	Média	10	Gabriel	Pré-requisito de tudo	—
Trilha BD (B1–B7)	Média	22	Matheus	Disciplina de Banco de Dados	a definir

Carga estimada por pessoa: Gabriel ≈ 64 h, Matheus ≈ 58 h, Francisco ≈ 56 h, Allan ≈ 50 h. Allan assume mais testes de sistema se a carga ficar desigual.

12. Riscos do projeto

O maior risco é concentrar conhecimento técnico em Gabriel; o plano mitiga isso exigindo que todos escrevam e expliquem testes.

Risco	Probabilidade	Impacto	Mitigação
Grupo de 4 recusado (limite de 3)	Média	Alto	Confirmar por escrito na S1; se recusado, redistribuir a trilha de BD
Dependência do líder técnico	Alta	Alto	Pair programming nas S4–S5; cada integrante demonstra 1 teste de cada tipo no ensaio
Incompatibilidade Java 25 / Spring Boot 4 com springdoc ou JaCoCo	Média	Alto	Validar na Fase 0; usar springdoc 3.x e JaCoCo $\geq$ 0.8.14
Docker indisponível em alguma máquina (Testcontainers)	Média	Médio	Rodar a suíte no GitHub Actions; Docker Desktop no Windows com WSL 2
Correções quebram o contrato e testes antigos	Média	Médio	Contrato versionado (v0 $\rightarrow$ v1) e regressão obrigatória no CI
Números não reproduzíveis na apresentação	Baixa	Crítico	Todos os números vêm de relatórios gerados pelo CI; nada digitado à mão
Atraso acumulado até o fim do semestre	Média	Alto	Marcos M1–M3 com revisão; S9 livre como margem

13. Demonstração e checklist de entrega

A regra 25 invalida qualquer resultado que não possa ser reproduzido ao vivo, então o ensaio precisa cobrir todos os pedidos listados pelo professor.

Roteiro de demonstração (preparar e ensaiar cada item)

Pedido do professor	Como demonstrar	Quem
Executar a suíte automatizada	<code>./mvnw verify</code> ou re-executar o workflow no GitHub	Gabriel
Executar um teste específico	<code>./mvnw test -Dtest=AddressApiTest#ct017*</code>	Qualquer um
Criar um novo caso de teste	Copiar o padrão da Fase 6 e alterar o dado de entrada	Qualquer um

Pedido do professor	Como demonstrar	Quem
Alterar um valor e provocar uma falha	Mudar o limite do CEP no teste para 7 e mostrar o FAIL	Allan
Mostrar a cobertura	Abrir <code>target/site/jacoco/index.html</code>	Matheus
Localizar um defeito	Issue D-xxx → caso CT-xxx → commit da correção → teste de regressão	Francisco
Explicar uma métrica	Taxa de aprovação e densidade de defeitos com interpretação	Matheus
Justificar um caso de teste	Valor-limite do CEP (7/8/9) ou tabela de decisão de pagamento	Allan
Demonstrar caixa-preta	Teste de API com validador OpenAPI	Allan
Demonstrar caixa-branca	Grafo de fluxo de <code>PaymentService.update</code> + branches no JaCoCo	Gabriel
Mostrar a análise estática	Painel do SonarCloud antes × depois	Francisco
Explicar a funcionalidade de maior risco	Matriz de risco: Pagamento/Pedido	Francisco

Checklist dos 10 entregáveis do enunciado

- ☐ 1. Código-fonte no repositório com `/src`, testes, `/docs`, `/reports`, `/config`, `README.md`
- ☐ 2. Plano de qualidade
- ☐ 3. Plano de testes
- ☐ 4. Catálogo de casos de teste (≥ 30)
- ☐ 5. Relatório de defeitos
- ☐ 6. Relatório de análise estática (antes e depois)
- ☐ 7. Relatórios de testes automatizados
- ☐ 8. Relatório de métricas
- ☐ 9. Relatório técnico final
- ☐ 10. Apresentação e demonstração prática
- ☐ Entregáveis de Banco de Dados (conforme resposta do professor)

14. Próximos passos (S1: 28/09 – 04/10)

- ☐ Reunião de kickoff de 1 h para aprovar este plano e a distribuição de papéis — todos

- ☐ Enviar ao professor de Qualidade a consulta sobre grupo de 4 e stack Java — Francisco
- ☐ Consultar o professor de BD sobre os entregáveis da disciplina — Matheus
- ☐ Fork, proteção de `main`, GitHub Project, labels e template de Issue — Gabriel
- ☐ Instalar JDK 25, Docker, IDE, Postman e DBeaver e subir a API localmente — todos
- ☐ Verificar se o Swagger sobe (DC-01) e congelar `config/openapi-v0.json` — Gabriel
- ☐ Primeiro rascunho dos 12 RF e 6 RNF — Allan

Fontes

- Enunciado: Projeto Final de Disciplina — Garantia de Qualidade de Software, prof. Christian Azonyetin (PDF do projeto)
- [Repositório da API sob teste](#)
- [springdoc-openapi FAQ — compatibilidade: springdoc 3.x para Spring Boot 4](#)
- [JaCoCo — histórico de versões \(suporte a Java 25\)](#)
- [SPARK-53987 — JaCoCo 0.8.14 para suporte oficial a Java 25](#)